Integrering av AI i en JavaScript-baserad audiovisuell musikapplikation

Att integrera AI i en Web Audio/Phaser/matter.js-applikation för generativ ambient musik öppnar möjligheter att skapa mer dynamiska och intressanta ljudmattor. Nedan diskuteras lämpliga AI-modeller, realtidsanalys av musikmönster, harmonigenerering, realtidsintegration i systemet, praktiska implementationstips med kodexempel samt inspirationskällor från existerande projekt och forskning.

AI-modeller och bibliotek för generering av ljudmattor

För att generera och optimera ambient ljudlandskap i realtid är **Magenta.js** (baserat på TensorFlow.js) ett utmärkt val. Magenta.js är en JavaScript-svit med färdiga ML-modeller för musikgenerering som kan köras direkt i webbläsaren med GPU-acceleration (Magenta.js). I synnerhet paketet `@magenta/music` inkluderar flera användbara modeller:

- **MusicRNN** – en RNN/LSTM-baserad modell för att generera melodier eller sekvenser. Magenta erbjuder varianter som *MelodyRNN*, *DrumsRNN*, *ImprovRNN* (melodi improvisation över ackord) och *PerformanceRNN* (för expressiva pianostycken) (magenta-js/music/README.md at master · magenta/magenta-js · GitHub). Dessa kan användas för att generera nya notsekvenser baserat på tränade stilar. Till exempel kan en MelodyRNN tränad på meditativ musik generera melodiska slingor i liknande stil.
- **MusicVAE** – en variational autoencoder för musik. MusicVAE kan generera eller interpolera kompletta musikfraser (t.ex. 4- eller 16-takters loopar) och även vara ackordstyrd för att hålla harmoni (magenta-js/music/README.md at master · magenta/magenta-js · GitHub). Den lämpar sig för att skapa variationsrika bakgrundsmattor; man kan t.ex. slumpmässigt sampla nya melodiska loopar kontinuerligt. (Magenta tillhandahåller förtränade checkpoints, t.ex. en 4-takters *trio* modell ~18MB som kan köras i webbläsaren (Magenta.js).)
- **TensorFlow.js** – underliggande ML-bibliotek som Magenta.js bygger på. Det möjliggör att köra både förtränade modeller och egenutvecklade nätverk i realtid i JavaScript (Magenta.js). Utöver Magentas modeller kan man med TF.js implementera egna neurala nät, t.ex. för förstärkningsinlärning (reinforcement learning) eller enklare nätverk för musikgenerering.
- **Magenta.js MusicVAE/MusicRNN API** – Biblioteket är relativt enkelt att använda. Man laddar en modell och anropar dess genereringsmetod. *Exempel:* För att generera en ny musikslinga kan man ladda en MusicVAE-checkpoint och sedan anropa `model.sample()` i en loop. Magentas dokumentation visar hur man kan skapa en oändlig ström av genererade musikfragment (Magenta.js):

```
const model = new mm.MusicVAE('URL_till_checkpoint');
const player = new mm.Player();
await model.initialize();
async function loop() {
```

```
  const [sequence] = await model.sample(1);      // Generera en ny NoteSequence
  player.start(sequence);                        // Spela upp sekvensen
  player.callbackAtSequenceEnd(loop);            // När sekvensen slutat, generera nästa
}
loop();
```

I koden ovan laddas en MusicVAE-modell och genererar sedan oändligt med nya sekvenser som spelas upp i följd (Magenta.js). (Magenta's `Player` använder Web Audio under huven, och kräver ofta Tone.js; alternativt kan man mappa sekvensen till sina egna oscillatorer/noder.)

- **Andra relevanta modeller** – Magenta har även specialiserade verktyg som *MidiMe* (för att anpassa en VAE-modell med lite egen data), *Piano Genie* (VQ-VAE som mappar få tangenttryckningar till 88-tangenter i realtid) (magenta-js/music/README.md at master · magenta/magenta-js · GitHub), *GrooVAE* (för humaniserade trumrytmer) etc. För ljudsyntes finns *GANSynth* och *DDSP* för att generera nya klanger (magenta-js/music/README.md at master · magenta/magenta-js · GitHub) (magenta-js/music/README.md at master · magenta/magenta-js · GitHub), men i en ambient Web Audio-app är det oftast enklare att använda vanliga synthljud och fokusera AI på notgenerering/styrning.
- **Förstärkningsinlärning (Reinforcement Learning)** – Utöver Magentas förtränade modeller kan man överväga RL om man vill att systemet själv lär sig optimera musiken över tid. Det finns inga out-of-the-box RL-musikbibliotek i JS, men man kan använda tf.js för att implementera t.ex. ett DQN-agent eller policy-gradient som justerar musikparametrar. Forskning har visat att RL kan användas för att uppnå global koherens i genererad musik – exempelvis genom att ett nätverk får utforska olika notsekvenser och belönas för långsiktig musikaliskt sammanhängande struktur (). (Ett avancerat exempel är *Bach DQN* där en agent med hjälp av ett djupt Q-nätverk genererar polyfon musik enligt musikregler () ().) I praktiken kan RL i webbläsaren vara komplext och tungt, så det är ofta bättre att använda förtränade modeller eller enklare heuristiker för realtidsändamål.

Sammanfattningsvis erbjuder Magenta.js/TensorFlow.js familjen mycket av det man behöver: förtränade RNN- och VAE-modeller för musikgenerering som kan köras i webappen. Dessa kan användas för att generera nya toner, slingor eller ackordföljder kontinuerligt under applikationens gång.

Realtidsanalys och förbättring av musikaliska mönster

För att AI:n ska kunna *analysera* och förbättra de genererade musikmönstren i realtid behövs mekanismer för att utvärdera musikalisk kvalitet eller variation pågående. Några metoder inkluderar:

- **Musikteoretisk analys**: Applikationen kan kontinuerligt extrahera information om de spelade tonerna (t.ex. via MIDI-liknande nothändelser från dina ljudnoder) och analysera mönster. Man kan detektera aktuell skala eller tonart genom att se vilka toner som är aktiva oftast, identifiera ackord (kluster av toner) samt mäta intervall

mellan toner. Genom att applicera musikteori-regler kan AI:n upptäcka t.ex. om ett dissonant intervall ofta uppstår eller om melodin börjar loopa monotont. Baserat på analysen kan systemet justera parametrar – t.ex. *quantize* tonerna till närmaste skalton om något hamnar utanför skalan, eller introducera en ackordväxling om samma ackord klingat för länge.

- **Mönsterigenkänning via ML**: Man kan använda en sekundär maskininlärningsmodell för att utvärdera musiken. Exempelvis skulle en RNN kunna tränas att förutse nästa ton i en "god" melodisk sekvens; om den faktiska musiken avviker för mycket från vad RNN förväntar sig (dvs. överraskningsmått), kan det indikera antingen kreativitet eller kaos – AI:n kan då besluta om det bör styras mot mer förutsägbara (koherenta) mönster eller inte. Alternativt kan man träna ett neuralt nät att bedöma "musikaliskt behag" eller spänning. Det finns forskning på att modellera musikspänning och släppa loss AI att optimera detta i realtid ((PDF) Adaptive music generation by reinforcement learning of ...), dock är sådana modeller avancerade. En enklare heuristisk variant är att definiera ett **”intressevärde”** för musiken baserat på t.ex. variation (för lite variation ger lågt värde) och harmoni (mycket dissonans ger lågt värde), och låta AI sträva efter att maximera detta värde över tid.
- **Förstärkningsinlärning för mönsterförbättring**: Med en definierad belöningsfunktion (t.ex. ovan nämnda intressevärde eller användarfeedback) kan en RL-agent pröva olika åtgärder i realtid för att förbättra musiken. *State* kan utgöras av det nuvarande musikaliska mönstret eller nodernas konfiguration, och *actions* kan vara att flytta en nod, byta en ton, eller ändra ett fysikaliskt attribut. Reward ges om musiken blir "bättre" enligt något kriterium. Over time kan agenten lära sig vilka förändringar som leder till t.ex. mer harmoniska eller varierade ljudmattor. Även om realtidsinlärning är svårt, kan en förtränad agent användas. Exempelvis visade Kotecha (2018) att en kombination av LSTM och DQN (deep Q-network) kan tränas att generera musik som följer musikaliska regler och ändå utforskar nya sekvenser (). I vår kontext kan man tänka sig att träna en agent offline på många simulerade ”bouncing node”-scenarier för att den ska lära sig optimera ljudmattan, och sedan köra agenten online för inferens i realtid.
- **Regelbaserad evolution**: Utan tung ML kan man också använda enklare algoritmer för att successivt förbättra mönstret. T.ex. genetiska algoritmer där flera varianter av nodkonfiguration genereras (mutationer) och utvärderas enligt en fitness-funktion (baserad på harmoni, rytmisk variation, etc.), varefter de bästa “överlever”. I realtid skulle detta kunna ske gradvis: systemet gör små slumpmässiga förändringar i musiken och behåller dem om de ger bättre resultat enligt någon heuristik. Detta ger en adaptiv utveckling av ljudmattan under körning.

Praktiskt bör analysen ske *kontinuerligt eller åtminstone per musikaliskt avsnitt*. Man kan exempelvis segmentera tiden i takter eller fraser och efter varje sådan analysera statistiken (fördelning av toner, ackordsekvenser, repetitioner) och besluta om justeringar behövs. Genom att kombinera domänkunskap (musikteori) med AI-modeller kan systemet hålla musiken både **koherent och levande**.

Algoritmer för harmoniska och intressanta tonkombinationer

Att säkerställa harmoniska tonkombinationer är centralt för meditativ ambient musik, så AI-komponenten bör hjälpa till att *identifiera* och *generera* toner som klingar väl ihop:

- **Skal- och ackordbegränsningar**: En grundläggande metod är att definiera en tonart eller skala (t.ex. D dorisk eller pentatonisk skala) som alla noder ska hålla sig inom. Då undviker man skärande toner utanför skalan. AI kan dynamiskt ändra skala eller tonart över tid för att skapa variation (t.ex. modulera från C-dur till G-dur), men alltid låta noderna stämma med aktuell skala. Likaså kan man gruppera noder i ackord – t.ex. tre noder utgör en C-dur treklang (C-E-G). Om partiklar triggar dessa tillsammans hörs ett ackord. AI:n kan planera ackordföljder (ex. en cirkel-femtedelsprogression eller annan generativ ackordföljd) så att när ackord byts, uppdateras nodernas toner därefter. På så vis uppstår *ackordsekvenser* som ger musikalisk riktning.
- **Datastrukturer för harmonilära**: Man kan programmera in kända harmoniska relationer, t.ex. använda en kostnadstabell för intervall (prim, kvint, kvart = låg "spänningskostnad", liten sekund eller överstigande kvart = hög kostnad). Algoritmen kan försöka minimera total "dissonanskostnad" mellan alla kombinationer av toner som spelas samtidigt. Om en viss nods ton orsakar dissonans med övriga vid kollisioner, kan AI:n justera den tonen till närmaste mer konsonanta ton. Sådana regler kan göras heuristiskt utifrån musikteori.
- **Ackordigenkänning och -generering**: AI kan lyssna på vilka toner som ofta samtidigt ljuder (t.ex. när flera partiklar aktiverar noder nära varandra i tiden) och försöka tolka det som ett ackord. Om ackordet är oklart eller dissonant kan systemet föreslå en justering. Generativt kan man ha en uppsättning **ackordmönster** (t.ex. I–IV–V–vi progressioner för dur, eller mer esoteriska ambient-ackord som sus2/maj7) och slumpa eller använda AI för att välja nästa ackord baserat på det föregående. En enkel algoritm är att bygga en Markov-kedja av ackord: t.ex. om nuvarande ackord är C-dur, så är sannolika nästa ackord F-dur eller G-dur, etc. AI:n kan lära sig en sådan kedja genom att analysera musik (korpusinlärning), eller man definierar den manuellt.
- **Neurala nät för harmonier**: Moderna ML-modeller kan också hantera harmoni. *ImprovRNN*-modellen i Magenta är t.ex. ackordstyrd – man ger den ett ackord (som text, t.ex. "Cmaj7") och den genererar en passande melodi för det ackordet (Connecting with Music Through Magenta.js). Detta innebär att modellen intern representation för vilka toner passar ett givet ackord. En sådan modell kan utnyttjas för att *upptäcka* vilket ackord en given tonföljd implicerar, eller för att *föreslå* melodier som är harmoniska. Man kan också använda *MusicVAE med ackord-conditionering*: Magenta erbjuder VAE-modeller som tar in ackordprogressioner för att generera polyfona slingor som passar dessa (magenta-js/music/README.md at master · magenta/magenta-js · GitHub). I realtid kan applikationen först bestämma en ackordföljd (antingen slumpad, förinspelad eller genererad av en enklare modell), och sedan använda en AI-modell för att fylla i melodiska eller texturmässiga detaljer som ligger harmoniskt rätt över dessa ackord.
- **Sammanfattningsvis**: Kombinationen av regler och AI ger bäst resultat. AI kan generera oväntade kombinationer som ändå låter bra, särskilt om den ges lite styrning. T.ex. om AI:n vet vilket ackord eller skala som gäller just nu, kan den *improvisera inom det ramverket* – detta tenderar ge "musikaliskt tillfredsställande resultat" (Connecting with Music Through Magenta.js). Om användaren slumpar fram toner fritt riskerar det bli kakofoni, men med ett AI-lager som justerar toner mot harmoniska kluster får man ett ambient soundscape där alla kombinationer av studsar låter genomtänkta. AI-algoritmen bör alltså både **identifiera** underliggande harmonik (vad

låter det som nu?) och **generera** nästa toner/ackord som skapar intresse utan att bryta den musikaliska logiken.

AI-integrering i realtid för att påverka noderna

Att integrera AI-komponenten i själva applikationens realtidsloop (Phaser + matter.js + Web Audio) kräver att AI:n kan påverka parametrar *medan* partiklarna studsar och ljud spelas:

- **Arkitektur**: Typiskt kan man köra AI-beräkningar i en separat tråd (t.ex. Web Worker) eller i *lagom takt* i huvudloopen för att inte störa rendering eller audiotråd. För ljudapplikationer är timing kritiskt; nätverksinferens bör helst ske asynkront. Magenta.js-modeller kör asynkront (de returnerar Promises) vilket underlättar integrationen. Man kan t.ex. varje gång en genererad sekvens är slut (eller var X:e sekund) trigga AI:n att beräkna nästa notsekvens, under tiden spelar appen vidare på redan bestämda toner.
- **Styrning av nodplacering**: AI kan bestämma hur noderna är utplacerade i grid-systemet. Kanske initialt placeras noder slumpmässigt, men AI:n kan justera deras positioner för att förändra rytmer eller tonkombinationer. T.ex. om en viss region i griden triggas för sällan kan AI sprida ut noderna mer jämt, eller gruppera vissa noder när den vill att flera toner ska klinga samtidigt (dvs. om AI vill skapa ett ackord, kan den klustra de toner så att en enda partikelkollision kan slå an dem nästan samtidigt). Realisationsmässigt kan man låta matter.js-noderna påverkas: AI:n kan applicera en liten kraft på en nod (flytta dess position) eller helt enkelt ändra kordinaterna ifall det inte stör fysikens stabilitet.
- **Styrning av tonhöjd/klang**: Varje nod har en viss ton (frekvens). AI:n kan i realtid ändra dessa toner för att följa en skala eller progression. Till exempel kan alla noders frekvenser transponeras upp några steg för att byta tonart, eller en viss nods ton kan justeras om AI:n märker att den skaver med resten. Det går att mappa en viss händelse (som att en partikel lämnar skärmen, eller att en viss tidsintervall passerat) till att AI:n byter ut några tonvärden för noderna. Eftersom Web Audio oscillatorer eller samplade ljud kan uppdateras on the fly, kan man göra detta utan att stoppa ljudet – nya kollisionshändelser helt enkelt triggar de *nya* tonerna.
- **Styrning av aktivering**: AI kan också direkt påverka vilka ljudhändelser som sker. Till exempel kan det “tända” eller “släcka” vissa noder temporärt – dvs. aktivera/inaktivera dem så att de antingen reagerar på kollisioner eller ignoreras. Detta kan skapa dynamik (en nod som tystnar i några sekunder för att sedan återkomma ger variation). AI:n skulle kunna besluta att "nu ska nod X vila en stund för att undvika att dess ton blir överexponerad". En annan aspekt är partikelgenerering: AI kan spawn:a nya partiklar eller ändra deras bana för att på så sätt påverka rytmen av trigging. T.ex. om musiken behöver tätare rytm, AI:n kan öka hastigheten på vissa partiklar eller släppa in fler bollar i systemet. All sådan interaktion måste göras försiktigt så det inte stör användarens framerate eller ljudets stabilitet.
- **Synkronisering och timing**: Viktigt är att AI-beslut integreras musikaliskt i tiden. Om AI t.ex. ska byta ackord (ändra flera noders toner), bör det ske i takt med musikalisk fras – kanske när en mätare tar slut eller på en downbeat – snarare än mitt i en ton. Man kan använda Web Audio API:s tidsfunktioner eller Phaser’s clock för att schemalägga sådana förändringar på rätt ögonblick. En metod är att ha en global **tempoklocka** eller tick i systemet (t.ex. 60 BPM för långsam ambient) och låta AI-åtgärder ske på takt-slag. Då upplevs ändringarna som en del av kompositionen snarare än slumpmässiga hopp.

- **Prestanda**: Om AI-modellen är stor (flera MB) kan initial laddning ta tid – se till att ladda/initialisera modeller tidigt (t.ex. innan starten eller under en loading screen). Vid realtidsanrop, undvik stora modeller för ofta. Exempelvis är en 4-bar MusicVAE (~18MB) hanterbar, men en 16-bar kan vara ~200MB+ vilket är opraktiskt i browser (Magenta.js). Håll er till mindre modeller eller använd vikt-kvantisering för att snabba upp. Ett tips är att använda Magenta.js i en WebWorker-tråd om möjligt (magenta-js/music/README.md at master · magenta/magenta-js · GitHub), så att huvudtråden (där Phaser/matter kör) inte blockeras under inferens. Skicka nödvändig data (t.ex. nuvarande sekvens eller state) till workern, låt den köra modellen och returnera resultat (nya notvärden/åtgärder) som sedan appliceras i huvudtråden.
- **Exempel på realtidsloop**: Pseudokod för att integrera AI-beslut i uppdateringsloopen:

```
// Antag: state innehåller aktuell musiksekvens eller nodinformation
function gameUpdate(delta) {
  updatePhysics(delta);
  playAudioEventsFromCollisions();
  // Varje X sekunder eller när en fras är klar:
  if (timeForAIAction()) {
     const currentPattern = extractPattern(nodes);
     aiModel.generateNext(currentPattern).then(newPattern => {
         applyPatternToNodes(newPattern);
     });
  }
}
```

  Här kallas AI-modellen periodiskt med nuvarande mönster (t.ex. sekvens av senast spelade toner eller nuvarande ackord) och när Promise:en *newPattern* kommer tillbaka appliceras det på noderna (t.ex. flytta noder eller ändra deras tonvärden). Detta sätt möjliggör att AI:n påverkar systemet i realtid utan att hålla på varje enskild collision-händelse.

Genom dessa mekanismer blir AI:n som en *medkompositör* som justerar spelplanen (nätet av noder) och instrumentinställningarna i farten, medan det underliggande fysikaliska systemet (studsande partiklar) står för den spontana rytmen och trigger själva noterna. Kombinationen ger både **struktur (från AI)** och **variation (från kaotisk fysik)** i ljudmattan.

Praktiska implementeringsförslag och kodexempel

För att komma igång med implementationen, här är några praktiska tips och enkla kodexempel:

- **Integration av Magenta.js**: Installera `@magenta/music` via npm eller lägg in en script-tagg. Initialisera sedan önskad modell. Exempel med en melodi-RNN:

```
import * as mm from '@magenta/music';
const melodyRNN = new mm.MusicRNN(
'https://storage.googleapis.com/magentadata/js/checkpoints/music_rnn/basic_rnn');
await melodyRNN.initialize();
// Skapa en startsekvens (NoteSequence) - kan också genereras från din nuvarande nodkonfiguration
const seedSeq = mm.sequences.quantizeNoteSequence(
```

```
    { notes: [ {pitch: 60, startTime:0, endTime:0.5} ], totalTime:
0.5}, 4);
// Generera en fortsättning på 8 takter med viss temperatur
(slumpmässighet)
const continuation = await melodyRNN.continueSequence(seedSeq, 32,
1.2);
console.log(continuation.notes);
```

Ovan laddas en enkel MelodyRNN-checkpoint. `continueSequence` används här för att generera fortsättningen på en kort frömelodi över 32 steg (8 takter i kvantiserad 4/4) med temperatur 1.2 för viss kreativitet. **Notera:** Istället för en statisk frösekvens kan du konstruera `seedSeq` dynamiskt utifrån vad som nyligen spelats i din app (d.v.s. låt AI:n fortsätta på det aktuella mönstret). Resultatet `continuation` är en Magenta *NoteSequence* som innehåller en lista av noter (med attribut som pitch, startTime, duration). Dessa noter kan du sedan mappa till dina noder/ljud.

- **Mappning av NoteSequence till noder**: Antag att din `continuation` innehåller t.ex. noter för en hel takt. Du kan då använda den för att omplacera/ändra noder:
  - Alternativ 1: **Schemalagd spelning** – ignorera fysikaliska studsar och spela upp sekvensen direkt via Web Audio. Du kan t.ex. iterera över `continuation.notes` och för varje note använda `audioContext.createOscillator()` eller liknande, sätta oscillatorns frekvens enligt `note.pitch` (MIDI-pitch 60 = C4 ~ 261.6 Hz), och schemalägga oscillatorn att starta vid `note.startTime` och stoppa vid `note.endTime`. Detta ger dig en AI-genererad melodi som spelas upp. (Magenta har även `mm.Player` som kan spela en NoteSequence automatiskt om du kopplar in en SoundFont eller synth.)
  - Alternativ 2: **Påverka spelet** – tolka NoteSequence som instruktioner för hur noderna ska ordnas. Exempelvis kan du låta varje *unikt* tonhöjdsvärde i sekvensen motsvara en nod i din grid och sätta nodens frekvens till detta värde. Eller om sekvensen har fyra fraser kan du placera ut fyra grupper av noder, där varje grupp representerar ackorden/tonerna i respektive fras. När partiklarna sedan studsar, triggar de dessa förvalda toner. I praktiken: sortera `continuation.notes` i kluster baserat på tidsfönster eller pitch, och låt AI:n bestämma att "i nästa sektion ska dessa toner vara aktiva." Du kan exempelvis ta de tre vanligaste tonerna i sekvensen och ställa in dem som de tre viktigaste nodernas frekvens (huvudackordet), etc.
- **Justera i realtid med RL eller heuristik**: För enkelhets skull, här är ett *pseudokod*-exempel på hur en enkel heuristisk förbättringsloop kan se ut utan att implementera full RL:

```
// Anta vi har en funktion för att beräkna 'harmoniScore' och
'varietyScore'
function evaluateMusicState(nodes) {
  let harmoniScore = computeHarmonicConsonance(nodes);
  let varietyScore = computePatternVariety(nodes, recentTriggers);
  return harmoniScore + varietyScore;
}
// Varje par sekunder, utvärdera och ev. justera en nod:
setInterval(() => {
  const scoreBefore = evaluateMusicState(nodes);
  // Prova en ändring:
```

```
    const randNode = pickRandom(nodes);
    const oldPitch = randNode.pitch;
    randNode.pitch = randomNearbyPitch(oldPitch);
    const scoreAfter = evaluateMusicState(nodes);
    if (scoreAfter < scoreBefore) {
       // ändringen var dålig, återställ
       randNode.pitch = oldPitch;
    }
}, 2000);
```

I detta pseudokodexempel ändras slumpmässigt tonhöjden på en nod då och då. Om det gör den samlade musiken sämre enligt våra utvärderingsfunktioner, ångras ändringen. Detta liknar *hill climbing* eller reinforcement learning i det lilla – över tid kommer nodernas tonhöjder optimeras mot högre poäng. Man kan utvidga detta med fler sorters ändringar (flytta position, aktivera/inaktivera nod, etc.) och mer sofistikerade utvärderingar.

- **Temperatur och stokasticitet**: När man använder modeller som MusicVAE eller RNN kan man ofta ange en *temperatur* eller liknande parameter som styr slumpmässigheten i genereringen. En högre temperatur ger mer variation och oväntade toner, lägre ger mer förutsägbara/melodiska resultat. För ambient musik vill man ofta ha relativt hög kreativitet men ändå inom ramen för harmoni. Experimentera med denna parameter för att få lagom balans. Till exempel kan man i mer aktiva sektioner höja temperaturen för mer spänning, och sänka den igen för att återgå till ett stabilt tema.
- **Lagring och återanvändning**: Om AI genererar något riktigt bra, kanske man vill lagra det (t.ex. i localStorage eller en global variabel) så att det kan repeteras eller utvecklas vidare senare, istället för att riskera att förlora det i nästa genereringscykel. Detta kan också fungera som ett "tema" som AI:n återkommer till – ett känt knep för att ge struktur är att ha återkommande motiv. AI kan således växla mellan *generera nytt* och *variation på tidigare tema* (en MusicVAE kan t.o.m. interpolera mellan två teman för en smooth transition).
- **Debugging och visualisering**: Det kan vara värdefullt att visualisera vad AI:n "tänker". T.ex. skriv ut de toner den väljer, eller rita ackordnamn på skärmen, så att man som utvecklare ser att "aha, nu bytte den till G-dur". Detta hjälper både vid utveckling och är också intressant för användare (edukativt). Magenta NoteSequence är lätt att konvertera till musiknotation eller pianorulle-visning om man skulle vilja inkludera en sådan för överblick.

Inspiration från existerande projekt och forskning

Det finns redan flera projekt och studier som kombinerar generativ musik, spelmotorer och AI – dessa kan ge inspiration och vägledning:

- **Magenta.js CodePen-exempel**: Magenta-teamet och communityn har skapat många exempel på generativ musik i webbläsaren. Bland annat:
  - *Endless Trios* – en demo som spelar en oändlig ström av auto-genererade pianotrior (Magenta.js). Den använder MusicVAE för att ständigt sampla nya 4-takters sekvenser som spelas upp i följd, och visar att långa, kontinuerliga kompositioner är möjliga att skapa i realtid.

- *Mindless Improv* – av Ian Simon. En AI som improviserar melodier över givna ackord i realtid (Magenta.js). Användaren kan byta ackord och höra AI:n reagera. Detta demonstrerar ett interaktivt sätt att få harmoniskt sammanhängande melodier (drivet av en ackord-konditionerad LSTM).
- *Latent Cycles* – av Tero Parviainen. Ett ambients experiment som utforskar musikaliskt latentutrymme (Connecting with Music Through Magenta.js). Fyra hörn representerar var sitt genererade mönster (t.ex. med ImprovRNN), och mellanliggande punkter ger interpolationer via MusicVAE. Användaren kan aktivera flera mönster samtidigt och höra hur de gradvis morfar och bildar komplexa texturer. Detta projekt är mycket relevant för ambient ljudmattor – det skapar *meditativa, evolverande* klanger genom AI och låter lyssnaren interagera med systemet.
- *Neural Drum Machine* – också av Tero Parviainen. Fokuserar på trumrytmer genererade av en RNN (magenta/magenta/models/music_vae/README.md at main · magenta/magenta · GitHub). Även om ert projekt är tonbaserat kan idéer hämtas om hur UI låter användaren styra slumpmässighet och hur rytmer kan skapas med ML.
- *Melody Mixer* och *Beat Blender* – experiment från Google Creative Lab + Magenta. **Beat Blender** låter användaren blanda trumloopar genom att dra en punkt i ett tvådimensionellt rum av beats genererade av en VAE (tränad på 3.8 miljoner trum-beats!) (Beat Blender by Creative Lab + Magenta - Experiments with Google). **Melody Mixer** gör liknande för melodier. Dessa visar hur ett AI-modell-latentutrymme kan göras navigerbart och lekfullt, så att användaren kan utforska olika musikaliska kombinationer. Även om ert system styrs av fysikaliska partiklar snarare än en direkt UI-dragpunkt, kan liknande "morphing" effekter uppnås genom att AI:n gradvis ändrar nodernas egenskaper.

- **Adaptive/dynamisk musik i spel**: Inom spelmusikområdet används algoritmisk musik för att anpassa sig till gameplay. Forskning har börjat titta på ML för detta. T.ex. visade Le Groux et al. (2010) hur *förstärkningsinlärning av musikalisk spänning* kunde generera adaptiv musik som följer spelets stämning ((PDF) Adaptive music generation by reinforcement learning of ...). Mer nyligen har man experimenterat med RL för att anpassa bakgrundsmusik i realtid baserat på spelhändelser eller publikens reaktion. För ert projekt, även om det inte är ett spel med nivåer, kan liknande tänk appliceras: AI styr musiken för att följa en viss "spänningskurva" över tid – t.ex. långsam uppbyggnad, kulmination, sedan avtoning, allt genererat autonomt. Artiklar om adaptive game music med ML kan ge idéer för hur man formulerar regler eller tränar modeller för att *respond to events* (här, kanske hastigheten på partiklar eller antal samtidiga kollisioner skulle kunna ses som "game state" som musiken anpassar sig efter).
- **”The Musician in the Machine” (Magenta blog, 2020)**: Beskriver ett projekt där ett team använde en *Piano Transformer*-modell för att komponera ambient musik (The Musician in the Machine). De tränade på en stor mängd ambientmusik och fick fram imponerande resultat. Även om transformermodellen de använde är tung för webben, är koncepten intressanta – de kombinerade miljöljud med neuralt genererade melodiska mönster för att efterlikna topp-ambient. Man kan dra lärdom om vikten av dataset och stil om man skulle träna egna modeller. (Magenta erbjuder exempel på att träna egna MusicVAE-modeller på ens data (magenta/magenta/models/music_vae/README.md at main · magenta/magenta · GitHub), om man vill att just *ens egen* ambient-stil ska läras in.)

- **Historiska generativa musikinstrument**: Det kan också vara givande att titta på pre-AI generativa verktyg, t.ex. *Brian Eno's generativa musikappar* eller *Laurie Spiegels "Music Mouse"*. Music Mouse (1986) var en ”intelligent instrument” som genererade förslagstoner när man spelade, baserat på musikteori – en tidig form av algoritmisk assistans. Tero Parviainen nämner att han portade Music Mouse till webben för inspiration (Connecting with Music Through Magenta.js). Dessa projekt visar hur regler och interaktion kan skapa meditativ musik utan ML, vilket ni sedan kan förstärka ytterligare med ML. Eno's verk å andra sidan betonar *system* där enkla regler ger upphov till komplexitet – ert fysiksystem är just ett sådant regelverk, och AI kan ses som ytterligare ett lager regler.

Sammanfattningsvis finns det många exempel som bekräftar att **kombinationen av AI och interaktiv generativ musik i webbläsaren är möjlig och spännande**. Genom att studera dessa projekt och kanske återanvända delar av deras kod eller idéer (många är öppen källkod) kan ni snabba upp er utveckling.

Källor:

- Google Magenta.js – JavaScript API för musikorienterade ML-modeller (Magenta.js) (magenta-js/music/README.md at master · magenta/magenta-js · GitHub)
- Magenta MusicVAE och MusicRNN (MelodyRNN, ImprovRNN m.fl.) (magenta-js/music/README.md at master · magenta/magenta-js · GitHub)
- Exempelkod för oändlig generering med MusicVAE (Magenta.js)
- Diskussion om att använda ackord-kontinuerade RNN-modeller för mer musikaliskt tillfredsställande resultat (Connecting with Music Through Magenta.js)
- Kotecha (2018), *Bach2Bach*: förstärkningsinlärning (DQN) för musikgenerering, ökad global koherens ()
- Beat Blender (Google Creative Lab + Magenta) – blending av trumloopar med en tränad VAE (Beat Blender by Creative Lab + Magenta - Experiments with Google)
- *Latent Cycles* av T. Parviainen – ambient generativt latent-utrymme med MusicVAE+ImprovRNN (Connecting with Music Through Magenta.js)
- Magenta-projektens lista över exempelapplikationer (Melody Mixer, Endless Trios, m.fl.)